PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Project: Vehicle Parking System

Item	**Student Entry**
Project ID	Vehicle Parking System
Group Number	10
Team Members	Roshan-PES1UG24CS387 S Mayur - PES1UG24CS391 Saima Kapoor-PES1UG24CS403 Rohan Jadar-PES1UG24CS379
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Version	1.0

DELIVERABLE 2 – REQUIREMENTS

2.1 Product Perspective

The Vehicle Parking System is a software-based parking management application that provides a centralized platform for customers, parking staff, and administrators. The system manages the digital parking workflow from vehicle registration and parking-slot reservation to QR-based entry validation, parking-session tracking, fee calculation, payment processing, and completion of parking sessions.

The system operates independently of physical parking hardware. Automated barriers, physical vehicle-detection sensors, and automatic number-plate recognition systems are outside the scope of the current system.

2.1.1 System Environment

The system is intended to operate as a web-based application accessed through standard desktop or mobile browsers. The software consists of a frontend interface, a backend application service, a database service and QR-processing components. QR validation uses camera-enabled devices for parking staff where applicable.

2.1.2 Constraints

The current implementation is limited to software-based parking management. Physical gates, barriers, vehicle sensors and ANPR are excluded. Payment processing is limited to a mock payment workflow. The initial release targets a single parking facility and does not include ERP or accounting integration.

2.1.3 Assumptions

Parking-slot information is maintained accurately by authorized administrators. Customers and parking staff have network access and compatible browser/camera capabilities where QR scanning is required. A valid reservation exists before a parking QR code is generated, and payment outcomes are returned by the mock payment component.

2.1.4 External Interfaces

User Interface: customer, parking-staff and administrator web interfaces. Software Interfaces: backend service to the database, authentication component, QR-processing libraries and mock payment component. Communication Interfaces: HTTP/HTTPS-based REST communication using structured request and response data.

2.2 Purpose

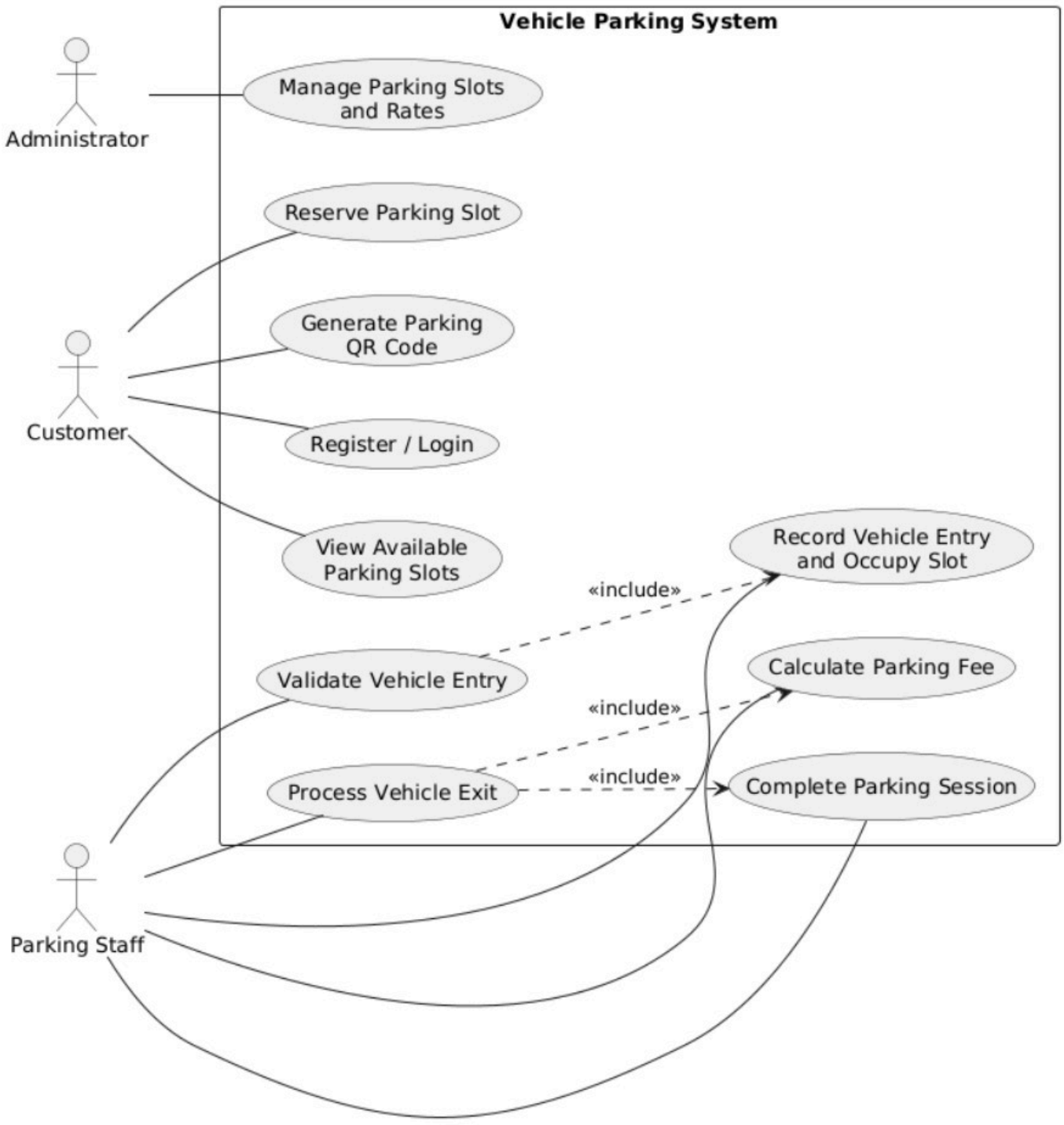
The primary objective of the **QR-Based Vehicle Parking System** is to deliver a robust and streamlined framework for supervising parking facilities via **QR-based entry and exit validation** protocols. This software empowers customers to monitor and book vacant spaces, create session-specific codes, oversee their usage history, and finalize billing; simultaneously, it equips operational personnel and management to coordinate infrastructure, space allocation, and financial records with precision.

2.3 Scope

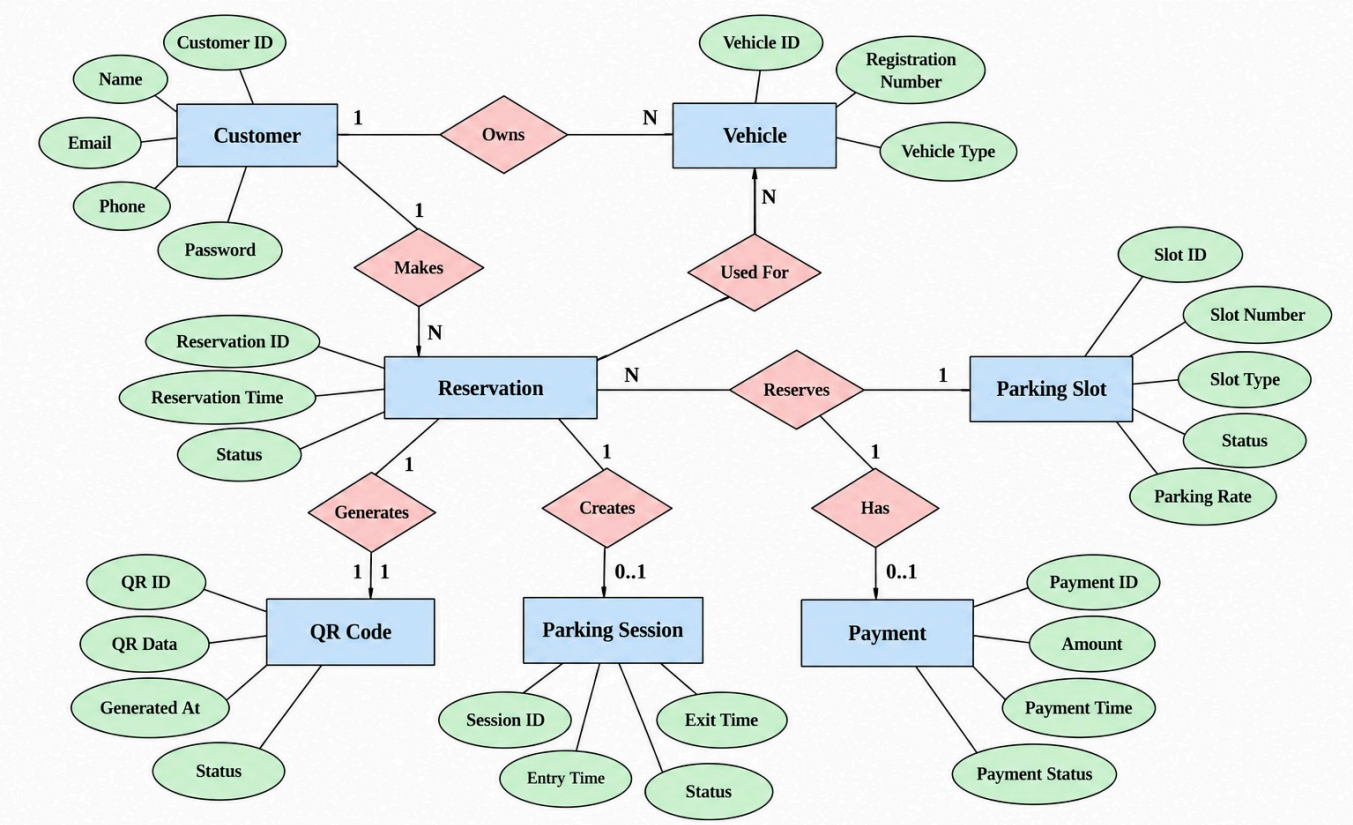
In Scope	Out of Scope
Customer/user registration and login	Automated physical parking barriers/gates
Vehicle registration and management	Automatic number-plate recognition (ANPR)
Real-time parking slot availability	Physical sensors for detecting vehicles
Parking slot selection and reservation	Integration with physical parking hardware
QR code generation for parking sessions	Multi-campus parking management
QR code scanning and entry validation	Integration with external ERP/accounting systems
Parking fee calculation and mock payment	Advanced AI-based parking demand prediction
Admin management of parking slots and parking rates	Valet parking automation
Parking history and transaction records	Navigation/GPS-based vehicle guidance

2.4 User Classes

User class	Privileges	Characteristics
Customer	Register vehicle, view available slots, reserve slot, generate and present QR code, view parking session, make payment, view parking history	Non-technical user; occasional or frequent parking usage
Parking Staff	Scan QR codes, validate vehicle entry/exit, view parking occupancy, update parking session status	Operational user; requires fast and simple interactions
Administrator	Manage users, vehicles, parking slots, parking rates, view transactions and parking reports	Higher privilege; trained system user



Visual representation of system interactions via Use Case Diagram



Entity-Relationship Diagram for Vehicle Parking System

2.5 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a user to register and log in using their basic details and vehicle registration number.	High	Functional test

FR-002	The system shall display the current availability of parking slots and allow a user to select an available slot.	High	Functional test
FR-003	The system shall allow an authenticated user to reserve an available parking slot for a registered vehicle.	High	Functional test

FR-004	The system shall generate a unique QR code for a confirmed parking reservation.	High	QR generation test
FR-005	The system shall allow parking staff to scan the user's QR code to validate the parking session at entry.	High	QR scanning test
FR-006	The system shall assign the selected parking slot to the vehicle and mark the slot as occupied after successful entry validation.	High	Functional test

FR-007	The system shall record the vehicle entry time and associate it with the corresponding parking session.	High	Database verification
FR-008	The system shall allow parking staff to scan the QR code at exit and retrieve the corresponding active parking session.	High	QR scanning test
FR-009	The system shall calculate the parking fee based on the recorded parking duration and configured parking rates.	High	Calculation test
FR-010	The system shall process a mock payment for the calculated parking fee and record the payment result against the corresponding parking session.	High	Integration test

FR-011	The system shall complete the parking session after successful exit processing and payment, and mark the corresponding parking slot as available.	High	Functional test
FR-012	The system shall provide an administrator interface to add, modify, remove and monitor parking slots and parking rates.	Medium	System test
FR-013	The system shall allow an authenticated customer to view their completed parking sessions and associated payment history.	Medium	Functional test
FR-014	The system shall allow an administrator to view parking-session, payment and slot transaction records for	Medium	System test

	operational monitoring.		

2.6 Non-Functional Requirements

ID	Category	Measurable requirement	Verification
NFR-001	Performance	The system shall display parking-slot availability within **2 seconds** for at least **95% of requests** under the defined test load.	Performance test
NFR-002	Availability	The system shall maintain at least **99% availability** during the defined operating period, excluding planned maintenance.	Operational review
NFR-003	Usability	A new user shall be able to complete registration and generate a parking QR code within 3 minutes without	Usability walkthrough

		assistance.	
NFR-004	Security	The system shall ensure that each generated QR code is associated with **one unique parking session** and shall reject invalid, expired, or already-used QR codes.	Security/functional test
NFR-005	Reliability	During testing, at least 99 of 100 simulated parking transactions shall preserve the associated session, vehicle, slot and transaction data correctly.	Reliability test
NFR-006	Maintainability	The system shall separate the frontend, backend, database, and QR-processing components so that an individual component can be modified without requiring changes to	Code inspection

		unrelated components.	

2.7 Security Requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall not be stored in plaintext.	Code/configuration review
SEC-002	Role-based access shall prevent customers from accessing staff/admin functions.	Authorization tests
SEC-003	The system shall validate and sanitize user supplied fields before processing.	Negative/security tests
SEC-004	Authenticated APIs shall reject expired/invalid credentials.	Security test
SEC-005	Sensitive payment information shall not be stored unnecessarily by the application.	Design review
SEC-006	Security-relevant events such as failed login and unauthorized access attempts shall be logged.	System test

2.8 Business Rules / Domain Rules

- Users are restricted to reserving **unoccupied parking slots** only.
- The reservation of a space that is **currently engaged or booked** is strictly prohibited.
- A **distinctive QR code** must be generated for every parking session, linking the specific vehicle to its reservation.
- The authority to authenticate codes and log vehicle movement is reserved for **designated facility personnel**.
- Access shall be denied for any **invalid, lapsed, or recycled** QR identifiers.
- System status for a slot transitions to **OCCUPIED** following a verified entry procedure.
- Slots revert to **AVAILABLE** status only upon the formal conclusion of the active session.
- Invoicing is determined by **total elapsed time and established pricing tiers**.
- A session cannot be finalized if the system lacks a **verified check-in timestamp**.
- Privileges to adjust slot configurations or financial rates are exclusive to **system administrators**.

2.9 Use Cases

Use Case ID	Use Case	Primary actor	Precondition	Outcome
UC-01	Register/Login	Customer	Application available	Authenticated session
UC-02	View Available Parking Slots	Customer	Application available	Available parking slots displayed
UC-03	Reserve Parking Slot	Customer	Logged in; available slot exists	Parking slot reserved

UC-04	Generate Parking QR Code	Customer	Valid parking reservation exists	Unique QR code generated
UC-05	Validate Vehicle Entry	Parking Staff	Staff authenticated; valid QR code presented	Vehicle entry validated
UC-06	Record Vehicle Entry and Occupy Slot	Parking Staff	QR code successfully validated	Entry time recorded and slot marked occupied
UC-07	Process Vehicle Exit	Parking Staff	Active parking session exists	Parking session retrieved and exit processed
UC-08	Calculate Parking Fee	Parking Staff	Active session with verified entry and exit	Parking fee calculated
UC-09	Process Payment	Customer	Parking fee calculated	Payment result recorded
UC-10	Complete Parking Session	Parking Staff	Exit processed and payment successful	Session closed and slot marked available
UC-11	Manage Parking Slots and Rates	Administrator	Administrator authenticated	Parking slots or rates updated
UC-12	View Parking History	Customer	Authenticated; customer has parking records	Customer parking history displayed

UC-13	View Transaction Records	Administrator	Administrator authenticated	Parking and payment records displayed

2.10 Validation Specification – Initial Planning

At the requirements stage, the validation specification is planning-focused. Students should explain how functional, security and non-functional requirements will eventually be validated, including planned penetration/security testing. This stage is not the final test report.

Requirement category	**Planned validation**	**Example**
Functional	Black-box functional tests	Valid/invalid parking slot reservation
Security	Authentication/authorization + security testing	Customer attempts to access admin functions
Performance	Load/response-time test	Concurrent parking slot availability requests
Usability	Task-based walkthrough	New user registers and reserves a parking slot
Reliability	Failure/retry scenarios	Payment failure and retry
Unit	Automated unit tests + coverage	Parking fee calculation

2.11 Project Plan

The project plan identifies and justifies the Agile lifecycle, tools, build/reuse components, work breakdown, effort estimates and project schedule.

Agile is selected because the system can be developed as a sequence of independently demonstrable features such as registration, slot reservation, QR validation, parking-session management, payment and administration. Sprint-based development allows the team to validate requirements early, incorporate feedback and continuously integrate testing with implementation. Each sprint produces a usable increment while preserving traceability from requirements to tests.

Work package	**Tasks**	**Owner**	**Estimated effort (person days)**	**Output**
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	Roshan/Mayur	4	Architecture/Design
WP3	Customer registration, login, and vehicle management	Roshan	6	Working feature
WP4	Parking slot availability and reservation module	Mayur	6	Working feature
WP5	Reservation, payment, and booking management	Saima	7	Working feature
WP6	QR e-ticketing, attendant scanner, and	Rohan	7	Working feature

	admin reporting			
WP7	Testing + CI/CD + security	All	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

2.12 Tool Stack

Activity	Selected tool	Evidence to submit
Frontend	React + Vite	Source code, build output
Backend	Node.js + Express	API implementation
Database	MongoDB	Schema and database evidence
Authentication	JWT + bcrypt	Authentication and authorization tests
QR processing	qrcode + html5-qrcode	QR generation/scanning evidence
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects	Backlog, sprint board and task ownership
CI/CD	GitHub Actions	Workflow logs for build and tests
Containerization	Docker	Dockerfile and build/run evidence
Testing	Jest + Supertest	Automated test results
Static analysis	SonarQube	Analysis report and remediation
API testing	Postman	Collection and test results
Security testing	OWASP ZAP	Security scan results
Design	Draw.io / PlantUML	UML diagrams

DELIVERABLE 3 – DESIGN

3.1 High-Level Architecture

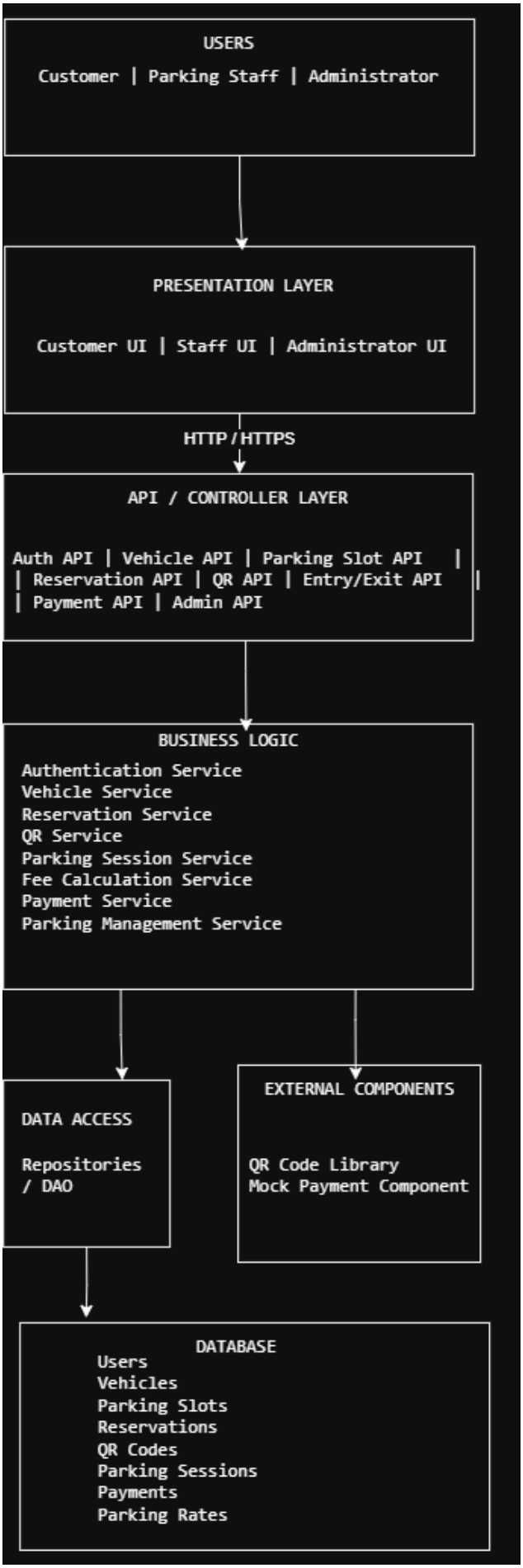

USERS
Customer | Parking Staff | Administrator
PRESENTATION LAYER
Customer UI | Staff UI | Administrator UI
HTTP / HTTPS
API / CONTROLLER LAYER
Auth API | Vehicle API | Parking Slot API |
| Reservation API | QR API | Entry/Exit API |
| Payment API | Admin API
BUSINESS LOGIC
Authentication Service
Vehicle Service
Reservation Service
QR Service
Parking Session Service
Fee Calculation Service
Payment Service
Parking Management Service
DATA ACCESS
Repositories
/ DAO
EXTERNAL COMPONENTS
QR Code Library
Mock Payment Component
DATABASE
Users
Vehicles
Parking Slots
Reservations
QR Codes
Parking Sessions
Payments
Parking Rates

Figure: High-Level Architecture of the Vehicle Parking System

Layer	Components	Responsibility
Presentation	Customer UI, Staff UI, Admin UI	Capture and display user interactions, parking availability, reservations, QR codes and parking-session information
API / Controller	Auth API; Vehicle API; Parking Slot API; Reservation API; QR API; Entry/Exit API; Payment API; Admin API	Request routing, input validation, authentication and authorization
Business Logic	Authentication Service; Vehicle Service; Reservation Service; QR Service; Parking Session Service; Fee Calculation Service; Payment Service; Parking Management Service	Business rules, reservation workflow, QR validation, session management, fee calculation and payment processing
Data Access	Repositories / DAO	Database interaction and persistence
Data	Customers; Vehicles; Parking Slots; Reservations; QR Codes; Parking Sessions; Payments; Parking Rates	Persistent storage of system records
External	QR Processing Library; Mock Payment Component	QR generation/validation and mock payment processing

Architecture flow: User → Web UI → REST API → Service Layer → Repository → Database.

The QR Service interacts with the QR processing library for QR generation and validation, while the Payment Service interacts with the mock payment component for payment processing.

3.2 Module Decomposition

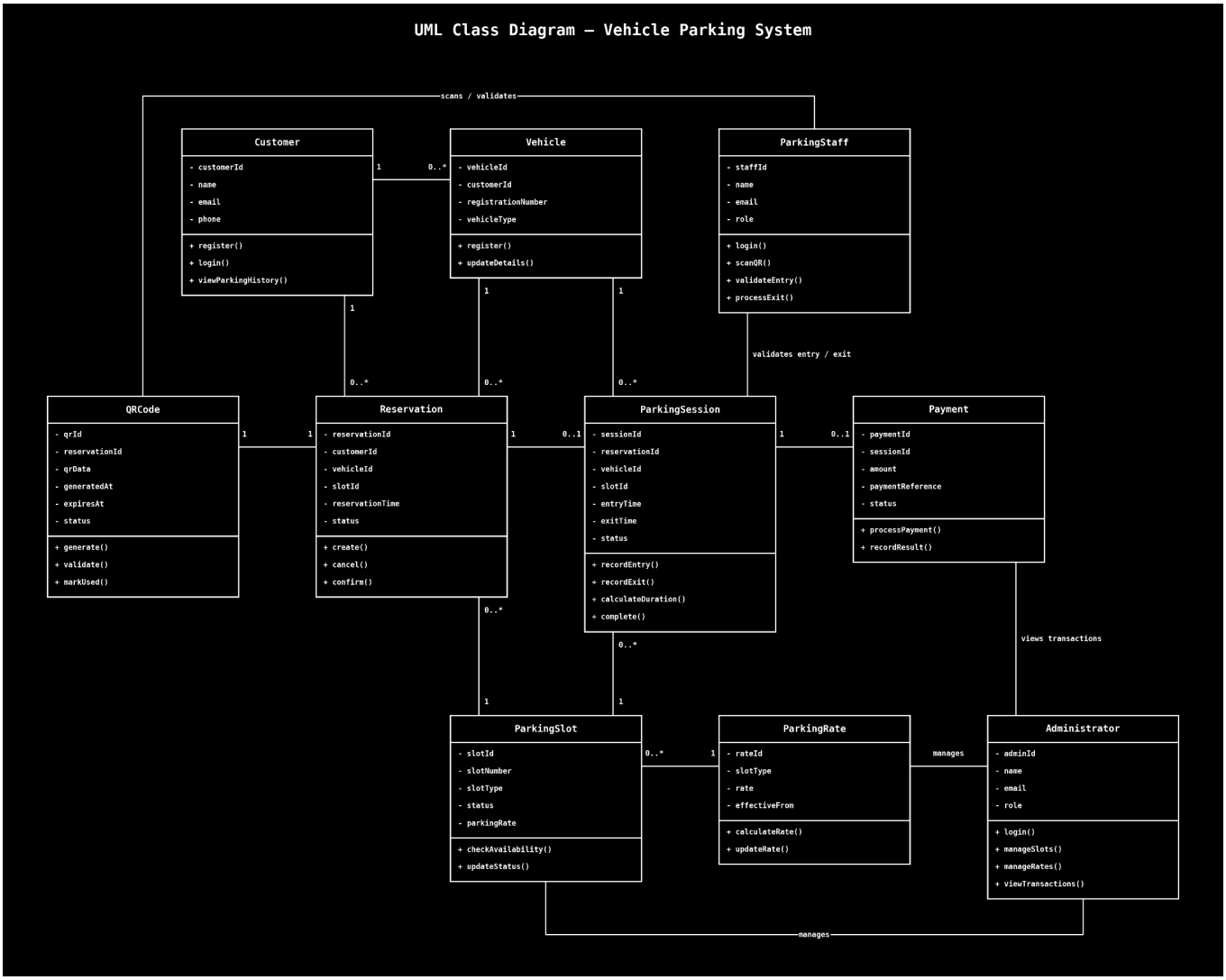
Module	Responsibilities	Depends on	Owner
Authentication	Registration, login, roles, session/token	Customer repository	Roshan
Vehicle Management	Register, view and manage customer vehicles	Authentication, Vehicle repository	Roshan
Parking Slot Management	Add, modify, remove and monitor parking slots	Parking Slot repository, Admin authentication	Mayur
Parking Reservation	Display available slots and create/manage reservations	Vehicle, Parking Slot, Authentication	Mayur
QR Processing	Generate and validate QR codes during entry and exit	Reservation, QR repository	Rohan
Parking Session	Record entry/exit, maintain session status and update slot occupancy	QR Processing, Parking Slot, Reservation	Saima
Fee Calculation	Calculate parking fee using duration and configured	Parking Session, Parking Rate repository	Saima

	rates		
Payment	Process mock payment and record payment result	Fee Calculation, Parking Session	Saima
Admin and Reports	Manage rates and view parking, session and transaction records	Parking Slot, Parking Rate, Payment, Parking Session	Rohan

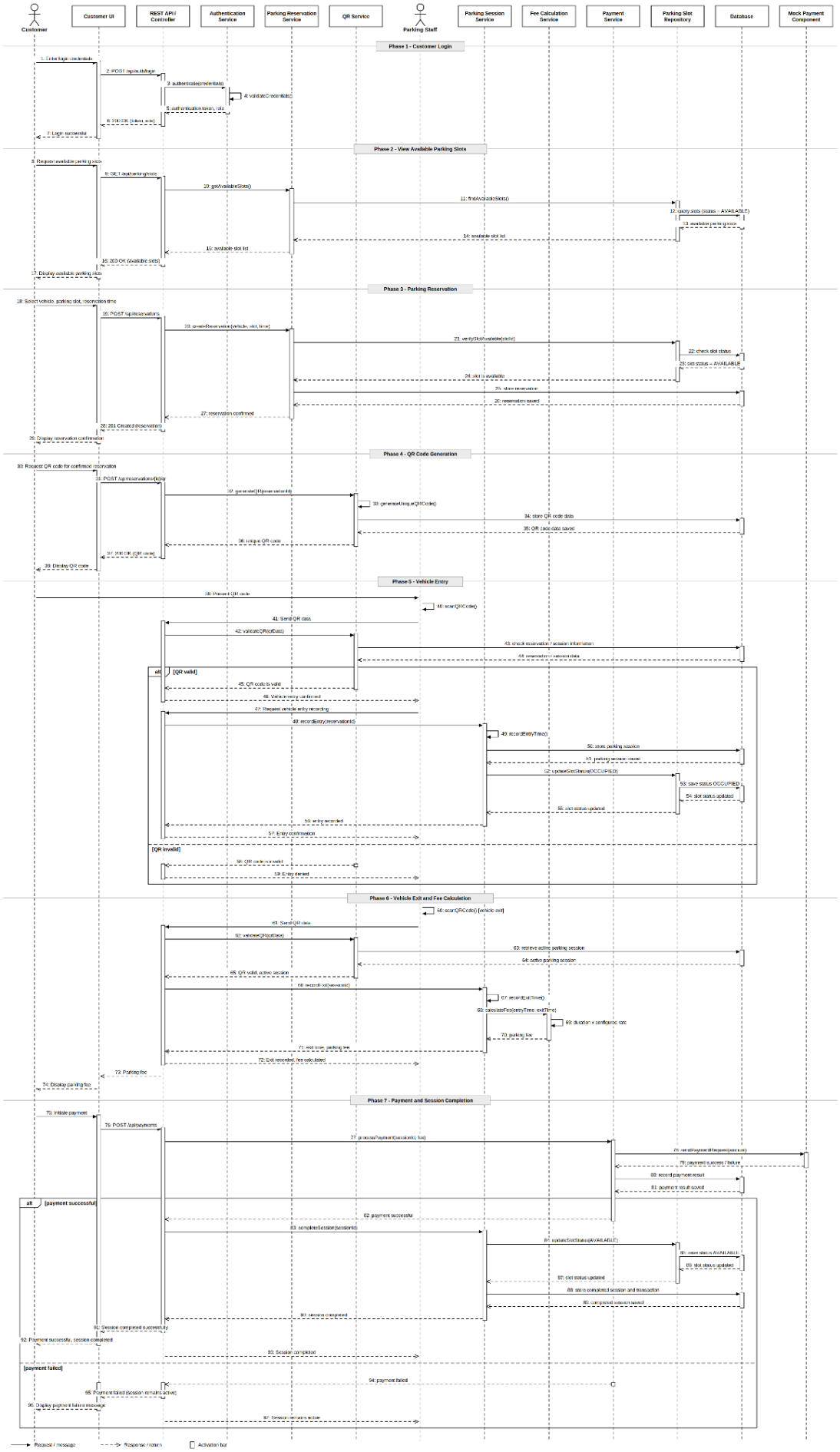
3.3 UML – Minimum Expected Set

The following UML models will be used to represent the Vehicle Parking System:

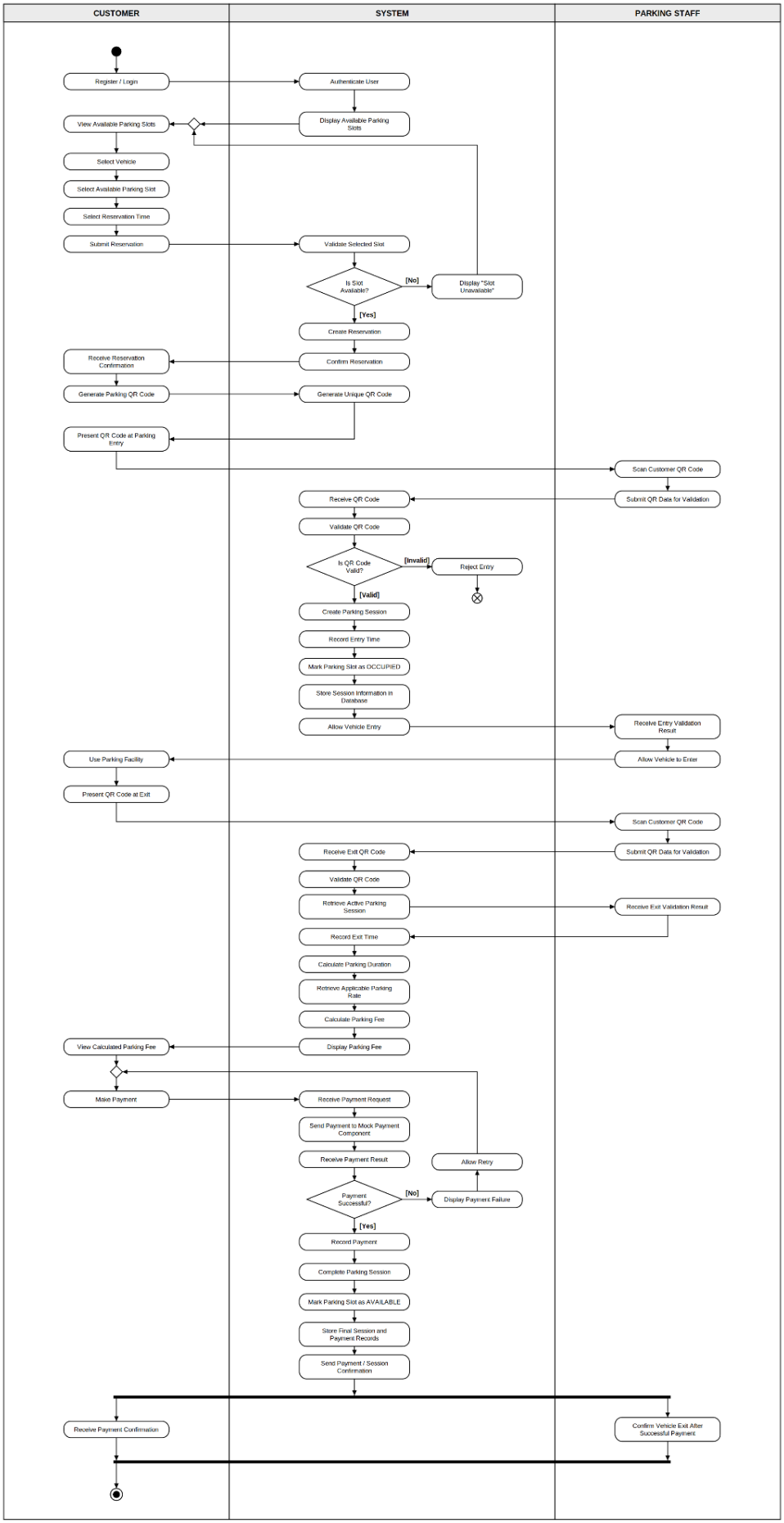
- **Use Case Diagram** – Represents the major actors and system interactions, including Customer, Parking Staff and Administrator.
- **Class Diagram** – Represents the major domain classes, their attributes, operations and relationships.
- **Sequence Diagram** – Represents the interaction sequence for major workflows such as parking reservation and vehicle entry/exit with payment.
- **Activity Diagram** – Represents the workflow of the parking process from reservation through entry, parking, exit, payment and session completion.
- **Component Diagram** – Represents the software modules, services, interfaces and their dependencies.


UML Class Diagram — Vehicle Parking System
scans / validates
Customer
- customerId
- name
- email
- phone
+ register()
+ login()
+ viewParkingHistory()
1
0..*
Vehicle
- vehicleId
- customerId
- registrationNumber
- vehicleType
+ register()
+ updateDetails()
ParkingStaff
- staffId
- name
- email
- role
+ login()
+ scanQR()
+ validateEntry()
+ processExit()
validates entry / exit
QRCode
- qrId
- reservationId
- qrData
- generatedAt
- expiresAt
- status
+ generate()
+ validate()
+ markUsed()
Reservation
- reservationId
- customerId
- vehicleId
- slotId
- reservationTime
- status
+ create()
+ cancel()
+ confirm()
ParkingSession
- sessionId
- reservationId
- vehicleId
- slotId
- entryTime
- exitTime
- status
+ recordEntry()
+ recordExit()
+ calculateDuration()
+ complete()
Payment
- paymentId
- sessionId
- amount
- paymentReference
- status
+ processPayment()
+ recordResult()
views transactions
ParkingSlot
- slotId
- slotNumber
- slotType
- status
- parkingRate
+ checkAvailability()
+ updateStatus()
ParkingRate
- rateId
- slotType
- rate
- effectiveFron
+ calculateRate()
+ updateRate()
manages
Administrator
- adminId
- name
- email
- role
+ login()
+ manageSlots()
+ manageRates()
+ viewTransactions()
manages

UML Sequence Diagram – Vehicle Parking Reservation and Parking Session



UML Activity Diagram – Vehicle Parking Workflow



3.4 Example Class Model

Class	Key attributes	Key operations
Customer	customerId, name, email, phone	register(), login(), viewParkingHistory()
ParkingStaff	staffId, name, email, role	login(), scanQR(), validateEntry(), processExit()
Administrator	adminId, name, email, role	login(), manageSlots(), manageRates(), viewTransactions()
Vehicle	vehicleId, customerId, registrationNumber, vehicleType	register(), updateDetails()
ParkingSlot	slotId, slotNumber, slotType, status, parkingRate	checkAvailability(), updateStatus()
Reservation	reservationId, customerId, vehicleId, slotId, reservationTime, status	create(), cancel(), confirm()
QRCode	qrId, reservationId, qrData, generatedAt, expiresAt, status	generate(), validate(), markUsed()
ParkingSession	sessionId, reservationId, vehicleId, slotId, entryTime, exitTime, status	recordEntry(), recordExit(), calculateDuration(), complete()
Payment	paymentId, sessionId, amount, paymentReference, status	processPayment(), recordResult()
ParkingRate	rateId, slotType, rate,	calculateRate(),

	effectiveFrom	updateRate()

3.5 API Definition

Method	Endpoint	Purpose	Input	Output
POST	/api/auth/register	Register a customer and vehicle	name, email, phone, password, registration_number, vehicle_type	customer ID, registration status
POST	/api/auth/login	Authenticate customer, staff or administrator	email, password	authentication token, role
GET	/api/vehicles	Retrieve vehicles registered by the authenticated customer	authentication token	vehicle list
POST	/api/vehicles	Register a vehicle for a customer	registration_number, vehicle_type	vehicle ID, registration status
GET	/api/parking/slots	Display current parking-slot availability	optional slot_type/status filters	slot ID, number, type, status, rate
POST	/api/reservations	Reserve an available parking slot	vehicle ID, slot ID, reservation time	reservation ID, reservation status
GET	/api/reservations/{id}	Retrieve reservation details	reservation ID	reservation details and status
POST	/api/reservations/{id}/qr	Generate QR code for a confirmed reservation	reservation ID	QR ID, QR data, expiry/status

POST	/api/entry/validate	Validate a QR code for vehicle entry	QR data/token	validation result, reservation/session details
POST	/api/sessions/{id}/entry	Record entry and occupy the assigned slot	session ID, validated QR	entry time, session status, slot status
POST	/api/exit/validate	Validate QR code and retrieve active parking session	QR data/token	session ID, entry time, exit eligibility
POST	/api/sessions/{id}/exit	Record vehicle exit	session ID, validated QR	exit time, calculated fee, session status
POST	/api/payments	Process mock payment for a parking session	session ID, amount, payment reference	payment ID, payment status
POST	/api/sessions/{id}/complete	Complete the parking session after successful exit/payment	session ID, payment status	session status, slot status
GET	/api/history	View customer's parking history	authenticated customer	previous sessions and payments
GET	/api/admin/transactions	View parking and transaction records	admin authentication	session, slot and payment records
POST	/api/admin/slots	Add a parking slot	slot number, type, rate	created slot
PATCH	/api/admin/slots/{id}	Modify or disable a parking slot	slot details/status	updated slot

PATCH	/api/admin/rates/{id}	Update parking rate	rate details	updated rate

3.6 API Contract Rules

- All API communication shall use **HTTP/HTTPS** with JSON request and response bodies.
- Authenticated endpoints shall require a valid authentication token.
- Role-based authorization shall restrict **Customer**, **Parking Staff**, and **Administrator** operations to their permitted functions.
- Request payloads shall be validated for mandatory fields, valid identifiers, permitted status values and appropriate data formats before processing.
- Every API shall return a defined HTTP status code for successful and unsuccessful operations.
- Resource identifiers such as customer ID, vehicle ID, reservation ID, QR ID and session ID shall be unique and stable.
- A parking slot reservation shall fail if the slot is unavailable at the time of the request.
- QR validation APIs shall reject invalid, expired or previously used QR codes.
- Entry and exit APIs shall operate only on valid parking sessions and authorized staff requests.
- Payment APIs shall record only the payment result and required transaction reference; sensitive payment information shall not be unnecessarily stored.
- Administrative APIs shall reject requests from customers and unauthorized staff.
- API contracts shall remain stable once dependent modules have been implemented; contract changes shall be version-controlled and reviewed.
- Error responses shall contain a consistent structure with an error code and descriptive message without exposing sensitive system information.

3.7 Database / Field Layout

Costumer:

Field	Length	Data Type	Description	Mandatory

customer_id	36	UUID/String	Unique customer identifier	Y
name	100	String	Customer name	Y
email	100	String	Unique login email	Y
phone	15	String	Customer contact number	Y
password_hash	255	String	Hashed customer password	Y

Vehicle:

Field	Length	Data Type	Description	Mandatory
vehicle_id	36	UUID/String	Unique vehicle identifier	Y
customer_id	36	UUID/String	Owner of the vehicle	Y
registration_number	20	String	Unique vehicle registration number	Y
vehicle_type	20	Enum/String	Car, bike or supported type	Y

Parking Slot:

Field	Length	Data Type	Description	Mandatory
slot_id	36	UUID/String	Unique parking-slot identifier	Y
slot_number	10	String	Displayed parking-slot number	Y

slot_type	20	Enum/String	Supported vehicle type/category	Y
status	20	Enum/String	AVAILABLE, RESERVED, OCCUPIED	Y
parking_rate	10,2	Decimal	Configured rate for the slot	Y

Reservation:

Field	Length	Data Type	Description	Mandatory
reservation_id	36	UUID/String	Unique reservation identifier	Y
customer_id	36	UUID/String	Customer making the reservation	Y
vehicle_id	36	UUID/String	Vehicle associated with reservation	Y
slot_id	36	UUID/String	Reserved parking slot	Y
reservation_time	—	DateTime	Time reservation was created	Y
status	20	Enum/String	Reservation status	Y

QR Code:

Field	Length	Data Type	Description	Mandatory
qr_id	36	UUID/String	Unique QR identifier	Y

reservation_id	36	UUID/String	Reservation associated with QR	Y
qr_data	500	String	Encoded QR token/data	Y
generated_at	—	DateTime	QR generation time	Y
expires_at	—	DateTime	QR validity expiry time	Y
status	20	Enum/String	ACTIVE, USED, EXPIRED, INVALID	Y

Parking Session:

Field	Length	Data Type	Description	Mandatory
session_id	36	UUID/String	Unique parking-session identifier	Y
reservation_id	36	UUID/String	Reservation used for session	Y
vehicle_id	36	UUID/String	Vehicle using the slot	Y
slot_id	36	UUID/String	Occupied parking slot	Y
entry_time	—	DateTime	Verified vehicle entry time	Y
exit_time	—	DateTime	Vehicle exit time	N
status	20	Enum/String	ACTIVE, COMPLETED	Y

Payment:

Field	Length	Data Type	Description	Mandatory
payment_id	36	UUID/String	Unique payment identifier	Y
session_id	36	UUID/String	Parking session being paid for	Y
amount	10,2	Decimal	Calculated parking fee	Y
payment_time	—	DateTime	Time of payment attempt	Y
payment_status	20	Enum/String	SUCCESS, FAILED, PENDING	Y
transaction_reference	100	String	Mock payment transaction reference	N

3.8 Design-to-Validation Update

Design Element	Validation Question	Planned Test
Role-based API authorization	Can a customer access staff/admin endpoints?	Authorization/security tests using customer credentials
Authentication API	Are invalid or expired credentials rejected?	Invalid-login and expired-token tests
Vehicle registration module	Can invalid or duplicate vehicle details be submitted?	Unit and negative functional tests
Parking-slot service	Can an unavailable slot be reserved?	Negative integration test
Reservation service	Can two requests reserve the same slot simultaneously?	Concurrent reservation test

QR-generation service	Is a unique QR generated for every valid reservation?	QR-generation unit/integration tests
QR validation service	Are invalid, expired or reused QR codes rejected?	Security and negative integration tests
Entry-session interface	Is entry recorded only after successful QR validation?	Entry workflow integration test
Parking-session state model	Can an invalid session state transition occur?	Negative integration tests
Exit validation interface	Can staff retrieve only an active parking session?	Exit validation system test
Fee calculation module	Is the fee calculated correctly from duration and configured rate?	Unit tests using boundary and normal-duration data
Payment interface	What happens when payment succeeds, fails or times out?	Mock payment success/failure/timeout tests
Session completion logic	Can a session be completed without verified entry/exit information?	Negative functional/integration test
Slot-status management	Does the slot change to OCCUPIED after entry and AVAILABLE after completion?	End-to-end system test
Database constraints	Can duplicate or invalid records be inserted?	Database/integration tests
Customer history interface	Are only the authenticated customer's sessions returned?	Authorization and data-isolation tests
Admin module	Can only administrators modify slots and rates?	RBAC and functional system tests
Logging module	Are failed login and unauthorized-access attempts recorded?	Security/system test
API performance design	Does slot availability meet the response-time requirement?	Load/response-time test
Modular architecture	Can individual modules be tested independently?	Unit tests with mocked dependencies

DELIVERABLE 4 – IMPLEMENTATION & TESTING

Implementation is expected to be iterative. The project flow connects Agile development, user tasks/user stories, code coverage, requirements traceability, team interaction, API/ABI stability and test-case development. Unit-test coverage is separately evaluated.

Implementation status: coding has not yet started. Source-code references, Git commits, pull requests, CI/CD execution, automated coverage reports and executed test results will be added as implementation progresses.

Member mapping: M1 = Roshan, M2 = Mayur, M3 = Saima, M4 = Rohan.

4.1 Agile Backlog Example

Story ID	User story	Acceptance criteria	Owner	Sprint
US-01	As a user, I want to register and add my vehicle so that I can use the parking system.	Valid account and vehicle details are accepted; duplicate vehicle registration is rejected.	M1	S1
US-02	As a user, I want to view available parking slots so that I can choose a suitable slot.	Available slots are displayed with their current status.	M2	S1
US-03	As a user, I want to reserve a parking slot so that a slot is assigned to	Only available slots can be reserved; successful reservation changes the	M2	S2

	my vehicle.	slot status.		
US-04	As a user, I want to generate a QR code for my parking session so that I can use it for entry validation.	A unique QR code is generated for every valid parking reservation.	M3	S2
US-05	As parking staff, I want to scan and validate a QR code so that I can authorize vehicle entry.	Valid QR codes are accepted; invalid, expired, or already-used QR codes are rejected.	M3	S3
US-06	As parking staff, I want to scan a vehicle's parking QR code at exit so that I can complete the parking session.	Valid exit QR is accepted; exit time is recorded and the slot is released.	M4	S3
US-07	As a user, I want the system to calculate my parking fee so that I know the amount to pay.	Fee is calculated correctly using parking duration and configured rates.	M4	S4

US-08	As a user, I want to make a payment for my parking session so that I can complete the parking process.	Successful payment completes the transaction; failed payment does not confirm the transaction.	M3	S4
US-09	As an administrator, I want to manage parking slots and rates so that the parking facility remains up to date.	Administrator can add, modify, disable slots, and update parking rates.	M1	S5
US-10	As an administrator, I want to view parking and transaction records so that I can monitor parking operations.	Correct parking sessions, payments, and slot records are displayed.	M2	S5
US-11	As a customer, I want to view my parking history so that I can review previous parking	Only the authenticated customer's records are displayed.	M1	S5

	sessions and payments.			

4.2 Git / GitHub Evidence

- Repository URL.
- Branch strategy used.
- Commit history showing individual contribution.
- Pull requests and review comments.
- Issues linked to user stories/tasks.

- Tags/releases used for sprint or milestone versions.
- README with setup, run and test instructions.

The guidelines state that every member must contribute to implementation and that Git commits will be checked.

4.3 CI/CD and Quality Evidence

Evidence	What students should show
CI pipeline	Build + automated tests run on push/PR
CD/deployment	Successful deployment to the chosen test environment
Code review	At least representative PRs with reviewer comments/approval
Static analysis	SonarQube or similar report and action on important findings
Containerization	Dockerfile and reproducible build/run evidence

4.4 Unit Test Coverage

Students must submit the automated unit test report and coverage report. Coverage should be reported for the implemented functional modules and should include the metric required by the course/tool configuration (for example, line and branch coverage).

Module	Example unit tests	Evidence
User & Vehicle Service	User registration, duplicate vehicle rejection, valid/invalid vehicle details	Jest/Pytest report + coverage
Parking Slot Service	Slot availability, slot reservation, occupied slot rejection, slot release	Test report + coverage
QR Service	QR generation, valid QR validation, invalid/expired QR rejection, duplicate QR usage	Jest/Pytest report + coverage
Parking Session Service	Session creation, entry/exit time recording, invalid session handling, session completion	Test report + coverage
Payment & Fee Service	Parking fee calculation, successful payment, failed payment, invalid transaction	Mock-based tests + coverage

4.5 Test Case Development

The test cases below cover unit, integration and system testing for the vehicle parking system. Actual results and pass/fail status are to be recorded after manual execution.

Test Case ID	Module	Description	Precondition	Steps	Test Data	Expected Result	Actual Result	Result

UT-01	Parking Fee Service	Calculate fee for a valid parking duration	Fee service is available	Provide parking duration and rate; calculate fee	2 hours, ₹30/hour	Total fee is ₹60	Not executed	Pending
UT-02	Parking Slot Service	Reject reservation of an occupied slot	Slot exists and is occupied	Attempt to reserve the occupied slot	Slot S01	Reservation is rejected; slot remains occupied	Not executed	Pending
UT-03	Parking Slot Service	Check availability of an available slot	Slot exists and is available	Request the availability status of the slot	Slot S02	System reports the slot as available	Not executed	Pending
UT-04	User Service	Reject registration with missing mandatory details	Registration service is available	Submit the registration form without a man	Missing vehicle registration number	Registration is rejected with an appropriate validation message	Not executed	Pending

				dato ry field				
IT-01	Reser vation & QR Servic e	Gener ate QR code after succe ssful reser vation	Custom er is logged in; slot is availabl e	Rese rve a slot, then requ est a QR code	Slot S02, valid custome r	Reservat ion is saved and a unique QR code is generate d	Not executed	Pending
IT-02	QR & Parkin g Sessio n Servic e	Reject an invali d QR code durin g entry	Entry validati on is availabl e	Scan an inval id QR code	Invalid QR token	Entry is denied and no parking session is created	Not executed	Pending
IT-03	Payme nt & Parkin g Sessio n Servic e	Handl e unsuc cessfu l paym ent	Parking session has a calculat ed fee and paymen t is availabl e	Initia te pay ment and simu late a faile d trans actio n	Failed paymen t respons e	Payment is not marked successf ul and the session is not marked as paid	Not executed	Pending
IT-04	Parkin g Sessio n Servic e	Record vehicle entry and occupy reserve	Staff authentic ated; reservati on and	Scan valid QR at entry; valida te	Valid QR for S02	Entry is recorded, parking session is active and S02	Not executed	Pending

		d slot after successful QR validation	QR code valid	session; update entry time and slot status		becomes OCCUPIED		
ST-01	Customer & Parking System	Complete a valid parking session	Customer is registered; slot is available	Reserve slot, generate QR, validate entry, record exit and complete session	Valid vehicle and reservation	Entry and exit are recorded, fee is calculated and the session is completed after successful payment	Not executed	Pending
ST-02	Security	Prevent customer from accessing administrator functions	Customer is logged in without administrator privileges	Attempt to access the parking-slot management endpoint	Customer token	Access is denied and no administrative changes are made	Not executed	Pending

UT-05	Parking Slot Service	Reserve an available slot	User logged in; slot is AVAILABLE	Reserve slot S02	S02 available	Reservation succeeds and S02 becomes RESERVED	Not executed	Pending
ST-03	Administrator	Manage parking slots and rates	Administrator authenticated	Add/ modify/disable a slot and update its rate	Slot S03, rate ₹40/hour	Changes are saved and reflected in the system	Not executed	Pending
ST-04	Administrator	View parking and transaction records	Administrator authenticated; records exist	Open transaction records	Existing session/ payment records	Correct parking-session and payment records are displayed	Not executed	Pending
ST-05	Customer	View parking history	Customer authenticated; completed parking records exist	Open parking history	Customer with previous sessions	Only the customer's own parking sessions and payments are displayed	Not executed	Pending

PERF-01	Performance	Measure parking-slot availability response time	Application running with defined test load	Send concurrent availability requests	Defined concurrent load	At least 95% of requests complete within 2 seconds	Not executed	Pending

4.6 Test Data Matrix

Scenario	Test Data	Expected Result
Valid registration	Valid customer details and vehicle registration number	Customer registration succeeds
Invalid registration	Customer details with a missing mandatory field	Registration is rejected
Valid slot selection	Existing available parking slot	Slot can be selected for reservation
Invalid reservation	Existing occupied parking slot	Reservation is rejected
Valid QR code	QR code associated with a confirmed parking session	QR code is accepted when presented by authorized staff
Invalid QR code	Random or malformed QR token	QR code is rejected
Expired QR code	QR code whose validity has expired	QR code is rejected
Reused QR code	Previously used QR code	QR code is rejected

Valid entry	Authenticated staff, valid QR code and available slot	Entry is recorded and the slot becomes occupied
Valid exit	Active parking session with a verified entry timestamp	Exit is processed for the corresponding session
Fee calculation	Recorded parking duration and configured parking rate	Fee is calculated according to the configured rate
Successful payment	Valid session and successful mock payment response	Payment is recorded as successful
Failed payment	Valid session and failed mock payment response	Payment is not recorded as successful
Session completion	Valid entry and completed exit processing	Session is closed and slot becomes available
Invalid session completion	Session without a verified check-in timestamp	Session completion is rejected
Security	Customer credentials used to access an administrator-only function	Access is denied and no administrative changes are made
Performance	Concurrent slot-availability requests under the defined test load	At least 95% of requests complete within 2 seconds
Reliability	Simulated system transactions involving sessions, vehicles, slots and transactions	At least 99% of simulated transactions preserve the required data correctly
Successful reservation	Authenticated customer and available slot S02	Reservation is created and slot status becomes RESERVED

Customer parking history	Authenticated customer with completed sessions	Only the customer's own history is displayed
Admin transaction records	Authenticated administrator with transaction records	Parking, payment and slot records are displayed

Code/File references are intentionally marked "Not yet implemented" because implementation has not started; planned test references will be finalized as the corresponding features are implemented.

4.7 Requirement Traceability Matrix

Requirement ID	Architecture Ref	Design Ref	Code/File Ref	Test Case ID	System Test ID
FR-001	Authentication	Registration and Login	Not yet implemented	UT-04	ST-01
FR-002	Slot Management	Slot Availability and Selection	Not yet implemented	UT-03	ST-01
FR-003	Reservation Module	Slot Reservation	Not yet implemented	UT-05	ST-01
FR-004	QR Module	QR Code Generation	Not yet implemented	IT-01	ST-01
FR-005	QR Module	Entry QR Validation	Not yet implemented	IT-02	ST-01

FR-006	Slot Management	Slot Assignment and Occupancy	Not yet implemented	IT-04	ST-01
FR-007	Session Management	Entry Time Recording	Not yet implemented	IT-04	ST-01
FR-008	Session Management	Exit Validation and Retrieval	Not yet implemented	ST-01	ST-01
FR-009	Payment Module	Fee Calculation	Not yet implemented	UT-01	ST-01
FR-010	Payment Module	Mock Payment Processing	Not yet implemented	IT-03	ST-01
FR-011	Session Management	Session Completion and Slot Release	Not yet implemented	ST-01	ST-01
FR-012	Admin Module	Slot and Rate Management	Not yet implemented	ST-03	ST-03
FR-013	History Module	Customer Parking History	Not yet implemented	ST-05	ST-05
FR-014	Admin/Reporting Module	Transaction Records	Not yet implemented	ST-04	ST-04
NFR-001	Performance Module	Response Time	Not yet implemented	PERF-01	PERF-01

NFR-002	System Operations	Availability Management	Not yet implemented	Planned	Planned
NFR-003	User Interface	Usability	Not yet implemented	Planned	Planned
NFR-004	QR Module	QR Uniqueness and Validation	Not yet implemented	IT-01, IT-02	ST-01
NFR-005	Session Management	Transaction Reliability	Not yet implemented	Planned	Planned
NFR-006	System Architecture	Modular Design	Not yet implemented	Planned	Planned
SEC-001	Authentication	Password Protection	Not yet implemented	Planned	Planned
SEC-002	Authorization	Role-Based Access Control	Not yet implemented	ST-02	ST-02
SEC-003	Input Validation	Input Validation and Sanitization	Not yet implemented	UT-04	ST-02
SEC-004	Authentication	Credential Validation	Not yet implemented	Planned	Planned
SEC-005	Payment Module	Payment Data Handling	Not yet implemented	IT-03	ST-01
SEC-006	Logging Module	Security Event Logging	Not yet implemented	Planned	Planned